

Octavio Morales

Patrick Hill

CS554- Tech Stacks

13 December 2024

Scenario 1: Implementing a Distributed E-Learning Platform

When designing the architecture of the e-learning platform to handle global scalability and high availability, I would take the approach of making subsections or a microservices approach to the architecture. Although it hasn't been worked with in the class, cloud computing such as Google Cloud or AWS in my opinion would be the best option to handle global scalability and high availability. Also, using systems that assist management and scaling of applications, like Kubernetes and Redis, as they can maintain low latency and fast responses globally. When it comes to video streaming to ensure high quality and minimal latency, there are many options one can choose from. There are many platforms tailored to different needs for the user. When it comes to recording, Open Broadcasting Service (OBS) is very useful, despite it being difficult to learn at first as it is consistent with low latency. However, I would recommend WebRTC as I've experienced similar results with less issues. The videos could be distributed through a global Content Delivery Network (CDN) like AWS CloudFront and displayed with AWS Media Services to ensure efficient video streaming and services. To handle the real-time collaborative features, such as chat or forums, I would use WebSockets to set up real-time communication. To handle the backend part of it, I would rely on Redis to ensure quick exchanges between messages. For storage of messages, especially shown in forums, I would use

MongoDB or Google Firebase (learned about this during Senior Design) to handle the database.

These two can also be used for managing course content, but from my own experience, I would prefer MongoDB. However, PostgreSQL may be far more beneficial for user progress and analytics due to it being a relational yet structured data model, but it is less flexible than MongoDB. Along with this, I would use redis to handle the caching for managing and analytics.

When it comes to sensitive data like user credentials, payment information and quiz results, encryption is very important. While transferring data, I would use Transport Layer Security (TLS) to encrypt the data, and can especially be seen with communication online. However, when it comes to data when kept in storage, AES-256 would be wise to use as it uses a 256 bit encryption algorithm, keeping everything private. With any program, there must be contingencies in place for disaster recovery and ensuring continuous operation during outages.

To monitor the program's health, I would recommend Grafana as it is better for handling data on a larger scale, so if the program starts to fail, you would be ready for it. Also, when deploying the program, it should be done in multiple regions to ensure backups in different locations. Lastly, if a data backup was needed, AWS Backup can be used to automate backups of data. However, a plan should be made to respond for all possible scenarios, so updates can be given to the users and an estimated time before the problem gets resolved.

Scenario 2: Developing a Social Networking Platform

When developing a social networking platform, there are a few resources I can use to develop real-time activity feeds and notifications. Like I mentioned in the previous scenario, WebSockets could be used for communication for real time updates and Redis Pub/Sub could be

used for notifications, broadcasting to every user. However, when it comes to devices other than desktops, Google Firebase Cloud Messaging (FCM) would be proven more useful to manage notifications on mobile devices as its efficient use has been seen from my prior experience with it. Now when it comes to the database used, I believe two can be used effectively to make the best product. PostgreSQL should be used for data like user profiles, any privacy settings, posts, and interactions due to its use in SQL and it being a relational database management system. Also, using MongoDB can be used for storing activity feed and notification as it is useful with dynamic data structures. For search and recommendation engines, Elasticsearch could be more useful than developing your own search engine, as it has features for autocorrect and typo correction. An algorithm can be made for a recommendation system, giving attributes to different profiles based on personality or location (like instagram), which would recommend to profiles with shared attributes. When it comes to data security and privacy for user accounts and communications, we can use OAuth 2.0 for user authentication and JWTs for session management. Like I mentioned in the previous scenario, I would use Transport Layer Security (TLS) to encrypt the data, and can especially be seen with communication online. However, when it comes to data when kept in storage, AES-256 would be wise to use as it uses a 256 bit encryption algorithm, keeping everything private. However, when it comes to malicious activity, it's important to keep audit logs (also encrypted) to keep users accountable, but it should follow all privacy laws. To scale a platform to accommodate a growing user base, once again mentioned in the previous scenario, Kubernetes can be used to distribute the data across different instances of the application. You can also take a different approach by using AWS Elastic Load Balancer to distribute the traffic effectively, especially if AWS CloudFront was used for CDN integration.

Now when it comes to moderation, two different paths can be taken, yet both can still be integrated. AI can be used for auto moderation and flag inappropriate content, like Google Perspective API, and prevent any harm early on. However, with manual moderation, a page for moderators can be used to review such flagged content, ban or unban users, or delete posts, so everything can be handled on a case by case basis. For the user interface, it's important to have all features, have everything run effectively, while also keeping it user friendly. As used before, React, Redux, Next.js and Apollo Client can be used to make the program, both frontend and backend. Lastly, Material-UI can be used for more features on frontend development.

Scenario 3: Creating an Event Management Web Application

When making a web application, there are many programs you can use for development, especially when it comes to managing events. In order to handle event creation and registrations efficiently, we can use a microservices architecture approach, or separating the services for everything like user management, event creation, etc. We could use AWS Elastic Load Balancer to make sure the traffic from the microservices can be handled effectively. We can even take it one step further by using Redis to cache event data that's been interacted with frequently, opening the possibility to reduce the traffic on the database during peak usage. In addition, we can use Stripe and Paypal to provide processing. Security and privacy is especially important when it comes to one's finances, and luckily both technologies provide security in their APIs. To keep track of events, it's important to integrate calendar functionality. There are countless APIs that can be used to achieve such, but Google Calendar API should be used as it is user friendly. For reminders, we can send notifications using Firebase Cloud Messaging and even schedule

them at no cost. As mentioned in the previous scenarios, either PostgreSQL or MongoDB can be used based on their benefits. However, when it comes to storing event details, user registrations, and payment information, PostgreSQL would be better as it is a relational database (as mentioned in the previous scenarios). To make sure that notifications are delivered reliably to users, Redis Pub/Sub could be used as it is capable of handling delivery at large scale. To generate analytics for event organizers, Metabase can be proven to be very effective, as not only can it generate reports for event organizers, their tools are also user friendly. As mentioned in previous scenarios, React and Material-UI can be used to make the user interface for event creation and management intuitive. Not only that, we can follow the “wizard” user interface method, to make it easier for users to create and customize events.

Scenario 4: Developing a Customer Relationship Management (CRM) System

When developing a CRM system for small businesses, it's important to have a reliable database to store customer information and interaction histories. As mentioned in the previous scenarios, we can store customer information and interaction histories using PostgreSQL as we can better organize the data's attributes by splitting the tables by different sets of data, like customers, interactions, etc. To implement the sales pipeline features, we can use React DnD to make the drag and drop menus for the sales pipeline and Redux for general management. However, for task management, it'll be easier to use a calendar to stay organized, to which Google Calendar API would be proven useful, using the PostgreSQL database to store everything. If I were to integrate email and SMS functionality into the system, I would use a service like AWS SES to handle emails and Nexmo for sending and receiving messages, as both

are very reliable when using data in bulk. To ensure data security and compliance with privacy laws, as mentioned in previous scenarios, we can use OAuth 2.0 for user authentication, AES-256 to encrypt data in storage and TLS to encrypt data in transit. There are laws we have to follow, like GDPR and CCPA, and we can follow them by implementing features for users, ensuring they have full control over it like data deletion and the user having to agree to data usage. We could use Metabase as a tool to generate data reports as mentioned in the previous scenario, but you could also achieve such using Javascript libraries like D3.js and Chart.js to create the dashboard and reports. As mentioned with the previous scenarios, we could design reporting tools for analyzing sales and customer data using React and Material-UI for the user interface while also using Redux and Next.js for the backend. To ensure the CRM system is scalable for growing businesses, we would deploy the CRM on a cloud network like AWS to allow resources to be used dynamically depending on the traffic, allowing businesses to grow with less issues.